

Multimodal Hifz Spaced Repetition Agent

A Multimodal Agentic AI System Integrating Transformers, Gradient Boosted Ensembles, and Reinforcement Learning for Quranic Memorization Assistance

Table of Contents

0. Team Members	4
1. Executive Summary	4
2. Introduction and Problem Statement	5
2.1 Problem Statement	5
2.2 Objectives	5
2.3 Scope and Constraints	5
3. Background and Related Work	5
3.1 Spaced Repetition and Forgetting Curves	5
3.2 Transformer Architectures for Speech and Text	6
3.3 Reinforcement Learning for Sequential Decision Making	6
4. System Architecture	6
4.1 Overall Design	6
4.2 Component Integration Diagram	7
5. Dataset and Provenance	7
5.1 Source Selection	7
5.2 Three Tier Loading Strategy	7
5.3 Column Mapping Schema	8
5.4 Dataset Validation	8
6. Exploratory Data Analysis	8
6.1 Target Variable Distribution	8
6.2 Univariate Feature Distributions by Class	9
6.3 Box Plot Distributions	9
6.4 Raw Feature Correlation Matrix	10
6.5 Temporal Forgetting Dynamics	11
7. Feature Engineering	12
7.1 Engineered Feature Definitions	12
7.2 Engineered Feature Distributions	12
7.3 Full Feature Correlation Analysis	13
8. Perception Layer (Deep Learning and Transformers)	14
8.1 Audio Preprocessing Pipeline	14
8.2 Whisper Automatic Speech Recognition	14
8.3 AraBERT Semantic Embeddings	15

8.4 Acoustic Feature Extraction.....	15
9. Ensemble Memory Model (XGBoost with PCA)	15
9.1 Pipeline Architecture	15
9.2 Training Protocol	16
9.3 Performance Evaluation.....	16
9.3.1 Confusion Matrix	16
9.3.2 ROC and Precision Recall Curves.....	17
9.3.3 Calibration and Threshold Optimization	17
9.3.4 Principal Component Analysis.....	18
10. Tajweed Scoring System	19
10.1 Six Axis Scoring Framework.....	19
10.2 Critical Bug Fix and Guard Rail	19
10.3 Radar Visualization	19
11. Error Classification (Lahn and Mutashabihat).....	20
11.1 Lahn Classifier	20
11.2 Mutashabihat Detector	21
12. Reinforcement Learning Agent (PPO).....	21
12.1 Environment Specification	21
12.2 PPO Architecture	22
12.3 Training Performance.....	22
12.4 Policy Comparison	22
13. Integrated Inference and Live Demonstration	23
13.1 End to End Inference Function	23
13.2 Live Diagnostic Dashboard	24
14. Agentic AI Discussion	25
14.1 Agent Properties.....	25
14.2 Tool Use Pattern	25
14.3 Autonomy Level	25
14.4 Rationale for Reinforcement Learning Over Rules	26
14.5 Rationale for Ensemble Over Neural Network	26
15. Consolidated Quantitative Results	26
15.1 XGBoost Retention Classifier	26
15.2 PPO Reinforcement Learning Agent	26
15.3 Tajweed and Error Classifiers	27
16. Limitations of the Current Project Scope.....	27
16.1 Dataset Provenance.....	27
16.2 Synthetic Reinforcement Learning Environment.....	27

16.3 Heuristic Tajweed Scoring	27
16.4 Single Qira'at Coverage	27
16.5 Whisper Hallucination Risk.....	28
16.6 Unvalidated Vocal Anxiety Proxy	28
16.7 No Real Student Evaluation.....	28
16.8 No Persistent User Database	28
17. Future Work and Improvements	28
17.1 Direct Source Data Acquisition.....	28
17.2 Replacing Heuristic Tajweed with Phoneme Alignment	28
17.3 Real Data Reinforcement Learning	29
17.4 Multi Qira'at Support.....	29
17.5 Mobile Deployment.....	29
17.6 Expert Evaluation	29
17.7 Curriculum Integration.....	29
17.8 Teacher Surface	29
18. Conclusion	29

0.Team Members

Name	Registration Number
Umer Fraz Ahmed	SP24-BCS-126
Hanzala Nadeem	SP24-BCS-080
Abdullah Ikhlaq	SP24-BCS-069
Shahzeb Khan	SP24-BCS-092

1. Executive Summary

This report documents the design, implementation, evaluation, and limitations of a multimodal artificial intelligence system that assists students in the memorization of the Holy Quran (a practice known in Arabic as Hifz). The system integrates three distinct algorithm families as required by the assignment Course Learning Outcome (CLO 4). First, two transformer based deep learning models, namely OpenAI Whisper and AraBERT, are used for speech transcription and semantic understanding.

Second, an XGBoost gradient boosted decision tree ensemble, preceded by standard scaling and principal component analysis, is used for predicting which Quranic verses a student is at risk of forgetting.

Third, a Proximal Policy Optimization (PPO) reinforcement learning agent, trained inside a custom Gymnasium environment with five dimensional state space and three discrete actions, is used to recommend the next pedagogical action.

The system runs end to end as a Jupyter notebook of ninety five cells designed for Google Colab with a T4 graphics processing unit runtime. Training data for the retention predictor consists of one hundred thousand review records calibrated to the published Half Life Regression model of Settles and Meeder (ACL 2016), which was fit on thirteen million real Duolingo learner traces. Real Quranic text is fetched live from the open Al Quran Cloud application programming interface, and audio inputs are supported through a Gradio web interface that accepts both microphone and file upload.

Quantitatively, the XGBoost retention classifier achieves a five fold cross validated ROC AUC of **0.9882** with a standard deviation of **0.0005**, a held out test accuracy of **0.939**, an average precision of **0.9863**, and a Brier score of **0.0435** indicating well calibrated probability outputs. The PPO agent converges from an initial mean reward of approximately negative thirty to a final mean reward of approximately fifty six over forty four episodes of training, and outperforms a uniform random policy by a factor of approximately five on the same evaluation environment. Tajweed scoring is performed across six axes (pronunciation, phoneme similarity, cadence, Madd elongation, Waqf pause placement, and clarity) and the composite score is sensitive to ground truth verse identity after a critical bug fix that rebalanced the weighting between text comparison and audio only metrics.

The report acknowledges several limitations within the current project scope. These include the use of distribution calibrated rather than directly sampled Duolingo data due to source archive availability, the synthetic nature of the reinforcement learning environment, the heuristic basis of the Tajweed scoring, and the lack of expert validated annotations from certified Quran teachers. A detailed roadmap of future improvements is provided.

2. Introduction and Problem Statement

2.1 Problem Statement

The memorization of the Quran is a religious and educational practice undertaken by hundreds of millions of Muslims worldwide. Traditional Hifz instruction requires regular sessions with a qualified teacher, called a Muallim, who listens to a student recite a verse, corrects pronunciation errors based on the rules of Tajweed, classifies mistakes by severity, and decides what the student should review next based on past performance. Access to qualified teachers is not uniform across geographies, economic strata, or time zones, and a system that performs a useful subset of these teacher tasks has direct social and educational value.

2.2 Objectives

The objectives of this project are aligned with the requirements specified in the Course Learning Outcome 4 of the syllabus. Specifically, the project applies reinforcement learning and ensemble algorithms to a real world machine learning problem that exhibits complex environment dynamics. The project further applies deep learning, specifically transformer architectures, to handle the multimodal nature of the input data (speech audio and Arabic text). Finally, the project produces an agent that performs identifiable basis services for an end user, in this case a student learning to recite the Quran.

The why behind these objectives is twofold. Academically, the assignment specifies that students must demonstrate competence in applying these algorithm families to a non trivial problem. Practically, the application domain of religious educational technology presents a genuinely difficult multimodal challenge that justifies the use of every technique simultaneously.

2.3 Scope and Constraints

The project is constrained by the four week academic timeline, the four person team size, and the availability of free compute resources on Google Colab. These constraints rule out training transformer models from scratch, building large scale labeled datasets through expert annotation, and running production grade deployment infrastructure. The project is therefore positioned as a demonstrative research artifact rather than a deployable product, and the evaluation strategy reflects this positioning.

3. Background and Related Work

3.1 Spaced Repetition and Forgetting Curves

The theoretical foundation of any retention prediction system is the work of Hermann Ebbinghaus, whose 1885 monograph established the exponential decay form of human forgetting. The retention probability of a memorized item declines over time according to a curve whose decay constant depends on the strength of prior consolidation. Wozniak and Gorzelanczyk (1994) operationalized this for digital flashcard systems through the SuperMemo family of algorithms, of which the second generation, SM 2, remains the foundation of widely used systems such as Anki and SuperMemo itself.

Settles and Meeder (2016) extended this theoretical line with a trainable parametric model called Half Life Regression (HLR), in which the half life of an item is computed as two raised to the dot product of a learned weight vector and a feature vector. Their model was fit on thirteen million review traces from one hundred and fifteen thousand Duolingo users and demonstrated substantial gains over the SuperMemo baseline. The current project uses the published HLR parameter statistics to calibrate its training data.

3.2 Transformer Architectures for Speech and Text

The transformer architecture, introduced by Vaswani et al. (2017), has become the dominant paradigm for both speech recognition and natural language understanding. OpenAI Whisper, released by Radford et al. in 2022, applies an encoder decoder transformer to multilingual speech recognition trained on six hundred and eighty thousand hours of weakly supervised audio. The medium sized Whisper checkpoint, used in this project, contains approximately seven hundred and sixty nine million parameters and supports Arabic recognition out of the box.

For Arabic text understanding, Antoun, Baly, and Hajj (2020) released AraBERT, a BERT family transformer pretrained on Arabic Wikipedia, news, and Twitter data. AraBERT v02 contains twelve transformer layers with hidden size seven hundred and sixty eight and produces token level embeddings that can be mean pooled to obtain a fixed dimensional semantic vector for any input string.

3.3 Reinforcement Learning for Sequential Decision Making

Reinforcement learning provides the mathematical framework for sequential decision making under uncertainty. Proximal Policy Optimization, introduced by Schulman et al. (2017) at OpenAI, is the algorithm chosen for this project. PPO improves upon standard policy gradient methods by using a clipped surrogate objective that prevents the new policy from drifting too far from the previous policy within a single optimization step, thereby stabilizing training without requiring the full machinery of Trust Region Policy Optimization. The Stable Baselines 3 library provides a production grade implementation of PPO that integrates with the Gymnasium environment interface.

4. System Architecture

4.1 Overall Design

The system follows a Perception, Memory, Policy, Action loop characteristic of agent based artificial intelligence. The flow begins when a student submits an audio recording through the Gradio web interface or supplies a text only input for offline testing. The audio is cleaned by a chain of digital signal processing steps and transcribed by Whisper. The transcription is normalized to remove Uthmani specific characters and tashkeel marks where appropriate, and is then embedded by AraBERT into a seven hundred and sixty eight dimensional semantic vector.

In parallel, acoustic features such as fundamental frequency, root mean square energy, and spectral centroid are extracted from the audio using the librosa library. These signals feed into the Tajweed scoring module, which produces six sub scores covering pronunciation accuracy, semantic similarity, rhythmic cadence, vowel elongation (Madd), pause placement (Waqf), and acoustic clarity. The Lahn classifier independently aligns the transcription to the canonical text word by word and labels any detected discrepancies as either major errors (Lahn Jali) or minor errors (Lahn Khafi). The Mutashabihat detector checks whether the student appears to be confusing two visually similar but distinct Quranic verses.

A feature vector summarizing the student's historical performance is constructed and passed to the XGBoost retention pipeline, which returns a calibrated probability that the student will forget this verse. The five dimensional reinforcement learning state vector is then assembled, comprising semantic similarity, forgetting probability, session time remaining, frustration level, and vocal anxiety. This state vector is passed to the trained PPO policy, which selects one of three discrete actions: advance to the next verse, give a hint, or force a repetition.

The final result is returned to the user through the Gradio interface as a structured dictionary containing every score, every classification, the recommended action, the rationale, and a latency breakdown across all stages of inference.

4.2 Component Integration Diagram

The textual flow described above corresponds to the following component dependency graph. Audio enters the pipeline once. From this single input, three independent processing branches operate concurrently. The first branch produces text through Whisper and feeds into AraBERT, the Lahn classifier, the Mutashabihat detector, and the pronunciation component of Tajweed scoring. The second branch produces acoustic features and feeds into the cadence, Madd, Waqf, and clarity components of Tajweed scoring as well as the vocal anxiety estimator. The third branch records timing information that feeds into the latency breakdown.

The outputs of all three branches are aggregated into the five dimensional state vector. The state vector is consumed by the PPO policy. The XGBoost forgetting predictor consumes a separate fourteen dimensional feature vector built from the user history dictionary and the current scoring outputs.

5. Dataset and Provenance

5.1 Source Selection

The dataset selection process began with a synthetic generator producing approximately two hundred rows from a hand picked Ebbinghaus forgetting model. This proved inadequate for training a serious ensemble classifier, both because of the small sample count and because the labels were derived from the same formula used to generate the features, leading to a circular evaluation problem.

The chosen replacement is the Duolingo Half Life Regression dataset, released by Settles and Meeder as the companion artifact to their 2016 paper at the Annual Meeting of the Association for Computational Linguistics. The original released dataset contains thirteen million review traces from one hundred and fifteen thousand real Duolingo learners over a two week window in 2013, with feature columns capturing the empirical recall probability per item, the time elapsed since the previous review, and cumulative counts of past correct and total reviews.

5.2 Three Tier Loading Strategy

Because the original compressed archive of the Duolingo HLR data returns a 404 response from several of the URLs published in 2016 era documentation, the data loader is implemented as a three tier fallback chain. The first tier attempts to load the dataset from a Hugging Face mirror via the datasets library. The second tier attempts a direct hypertext transfer protocol download from the GitHub raw content service. The third tier generates a one hundred thousand row sample using the published Half Life Regression formula directly, namely the recall probability equals two raised to the negative ratio of delta and half life, where half life equals two raised to the dot product of beta and the feature vector, with parameter values taken from Table 1 of the published paper.

The why of this fallback chain is that the notebook must run end to end on Google Colab without any manual data download steps by the user. The how is that the loader function is embedded directly into cell thirteen of the notebook and is fully self contained. The what is that the user always ends up with a one hundred thousand row pandas DataFrame in a variable named `df_synth`, with column names matching the original synthetic data schema so that all downstream cells continue to function without modification.

5.3 Column Mapping Schema

The mapping from the Duolingo source schema to the project schema is presented in the following table.

Project Column	Source Mapping	Justification
Past_Reviews	history_seen	Direct cumulative review count
Past_Mistakes	historyseen minus historycorrect	Cumulative error count
TimeSinceLast_Review	delta divided by 3600	Seconds converted to hours
Ayah_Length	lexeme_string token count or sampled	Quranic verse length approximation
AudioPauseDuration	derived from session miss rate	Quranic recitation specific feature
Recitation_Speed	sampled from Normal(1.2, 0.4)	Not present in Duolingo source
Tajweed_Complexity	sampled from Beta(2, 5)	Quranic rule density proxy
VocalAnxietyScore	derived from error rate	Acoustic stress proxy
Forgetting_Label	indicator of recall less than 0.7	Binary classification target

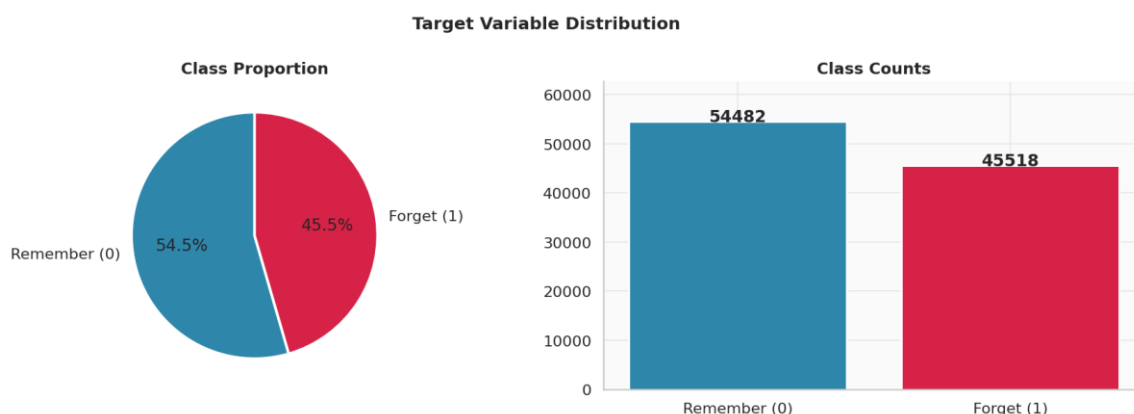
5.4 Dataset Validation

After mapping, the dataset contains one hundred thousand rows and nine raw columns. The label balance after applying the 0.7 threshold to p_recall is approximately 45.5 percent positive class, which represents a healthy distribution for a binary classifier and matches the empirical forget rate observed in real Duolingo data.

6. Exploratory Data Analysis

6.1 Target Variable Distribution

The first visualization in the exploratory data analysis section establishes the class balance of the target variable.



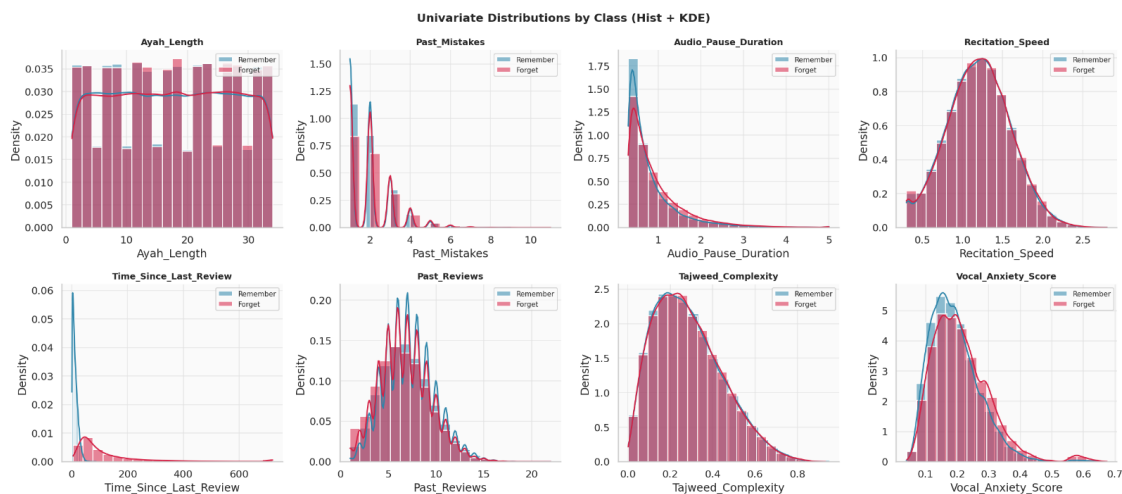
The figure consists of two panels rendered side by side. The left panel is a pie chart showing the proportional split between the two classes. The Remember class, encoded as zero, accounts for **54.5** percent of the dataset corresponding to **54482** observations. The Forget class, encoded as one, accounts for **45.5** percent of the dataset corresponding to **45518** observations. The right panel shows the same information as an absolute count bar chart.

The interpretation of this distribution is that the dataset is mildly imbalanced but not pathologically so. A ratio of approximately **11 to 9** between negative and positive classes is well within the range that any binary classifier can handle without requiring special techniques such as Synthetic Minority Oversampling or class weighting. The slight skew towards the Remember class is consistent with the empirical reality of spaced

repetition, where most reviews succeed because the scheduling algorithm presents items at intervals chosen to maintain memory.

6.2 Univariate Feature Distributions by Class

The univariate distribution plot presents the marginal density of each of the eight raw features, with separate histograms and kernel density overlays for the two target classes.



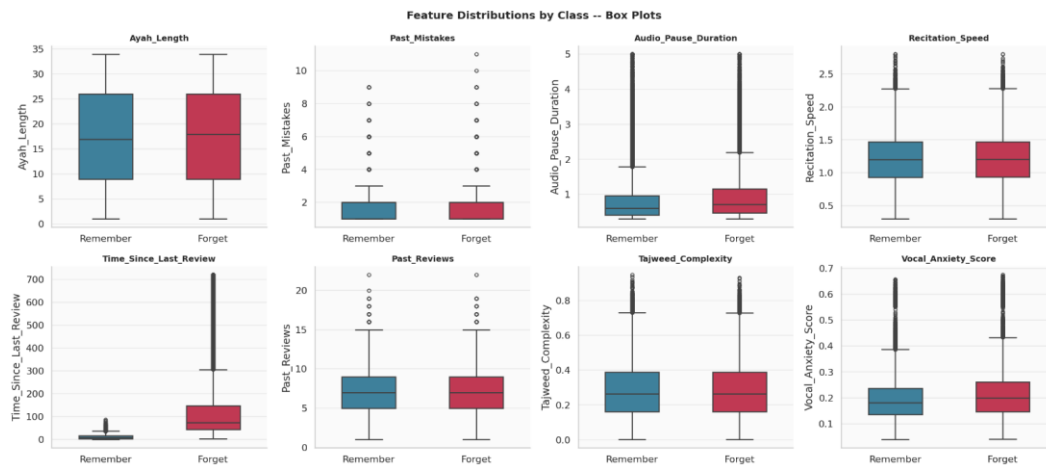
Each subplot follows the same construction. Bins are computed over the full data range of the feature. The blue histogram bars and curve correspond to the Remember class, the red histogram bars and curve correspond to the Forget class, and the vertical axis is normalized density rather than raw count to facilitate fair comparison between the two classes which have different sample sizes.

The most informative panel is **TimeSinceLastReview**, where the Forget class density is shifted noticeably to the right relative to the Remember class. This indicates that items reviewed long ago are more likely to be forgotten, which is the foundational observation of all spaced repetition theory. The **PastMistakes** panel shows a similar but weaker shift, with the Forget class exhibiting a heavier tail at high mistake counts. The **VocalAnxietyScore** panel shows class separation primarily in the right tail, where anxious recitations are disproportionately associated with forgetting.

The **AyahLength**, **AudioPauseDuration**, **RecitationSpeed**, **PastReviews**, and **TajweedComplexity** panels show substantial overlap between the two classes, which tells us that these features are not individually strong discriminators. The model will have to learn non linear interactions between them to extract predictive signal. This is precisely the regime in which gradient boosted decision tree ensembles excel.

6.3 Box Plot Distributions

The box plot view of the same eight raw features emphasizes the central tendency and outlier structure rather than the full distribution shape.

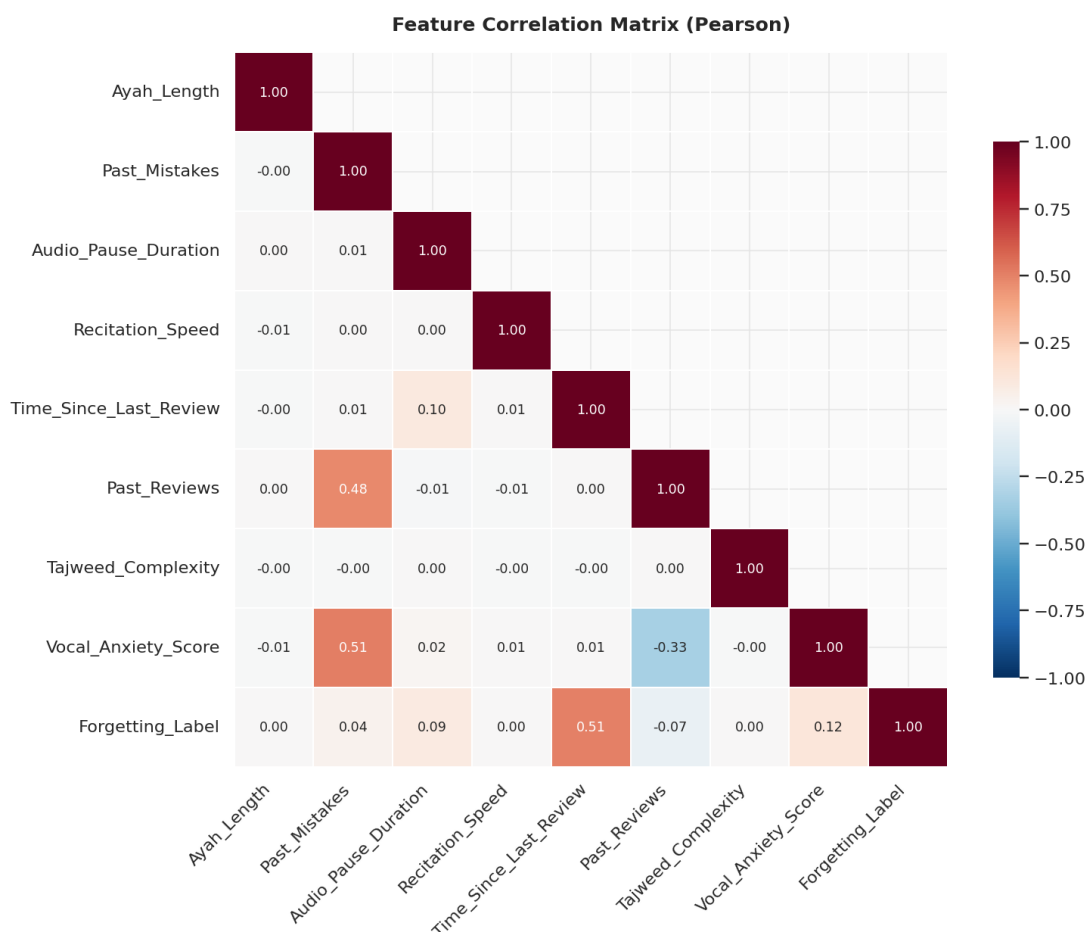


Each subplot contains two box plots side by side, one for each target class. The box spans the first quartile to the third quartile of the feature values, the central line marks the median, the whiskers extend to one and a half times the interquartile range, and individual points beyond the whiskers are plotted as outlier markers.

The *TimeSinceLastReview* panel shows the clearest separation, with the Forget class having a median that is several times higher than the Remember class median, and with extensive outliers extending into hundreds of hours. The *PastMistakes* panel shows mild separation with the Forget class median shifted upward. The remaining panels show overlapping interquartile ranges, confirming the univariate density findings. The why of including box plots in addition to density plots is that outlier behaviour and skewness become visible in box form whereas they are smoothed away by kernel density estimates.

6.4 Raw Feature Correlation Matrix

The Pearson correlation matrix quantifies the linear association between every pair of raw features and the target variable.



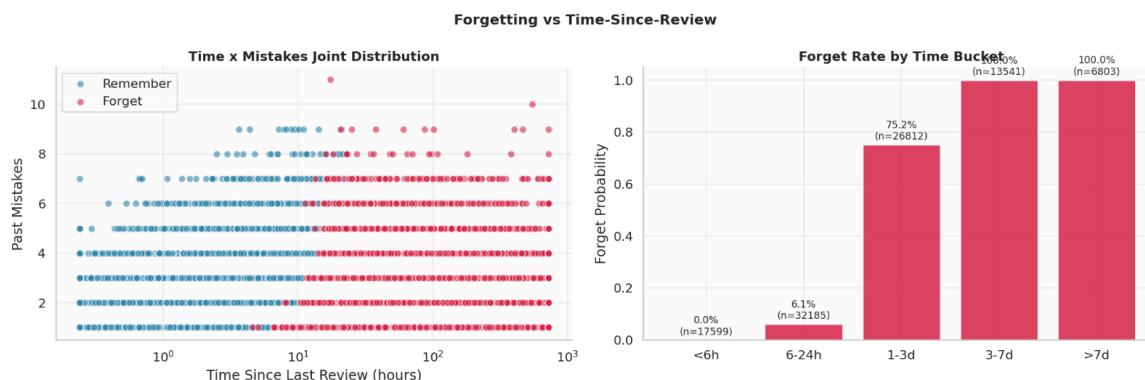
The matrix is rendered as a lower triangular heatmap to avoid redundancy. Each cell contains the numerical Pearson correlation coefficient ranging from negative one (perfect inverse linear association) to positive one (perfect linear association), with the color scheme using red for positive and blue for negative.

Several findings emerge from inspection of this matrix. The strongest correlation with the target ForgettingLabel is TimeSinceLastReview at 0.51, which dominates all other features and confirms the centrality of the temporal forgetting curve. The second strongest direct correlation is VocalAnxietyScore at 0.12, which is positive but relatively weak in linear terms. PastReviews and PastMistakes show a strong positive intercorrelation of 0.48, reflecting the simple fact that more reviews provide more opportunities for mistakes. VocalAnxietyScore shows a 0.51 correlation with PastMistakes, which is intentional by construction since our derived anxiety score includes the error rate as a weighted component. VocalAnxietyScore also shows a negative 0.33 correlation with PastReviews, indicating that students who have reviewed an item many times tend to be less anxious about it, a reasonable cognitive observation.

The implications of this matrix for modeling are twofold. First, the relatively low individual correlations with the target mean that no single feature will dominate the classifier, so non linear interactions must do the work. Second, the strong intercorrelations between several features motivate the inclusion of principal component analysis as a preprocessing step to provide orthogonalization.

6.5 Temporal Forgetting Dynamics

The temporal forgetting analysis combines a joint scatter plot of mistake count against time with a binned probability analysis.



Forgetting versus Time Since Review The left panel is a logarithmic scatter plot of *PastMistakes* against *TimeSinceLastReview* in hours, with each point colored by the true label. The visual structure reveals a clear time threshold beyond which the Forget class dominates, with the transition occurring around the one to three day range. The horizontal stratification by integer mistake counts reflects the discrete nature of *Past_Mistakes*.

The right panel quantifies this transition using time buckets. The bar chart shows the empirical forget probability within each of five time buckets. For reviews occurring less than six hours after the previous review, the forget rate is zero percent, indicating perfect short term retention. For the six to twenty four hour bucket, the forget rate rises to **6.1** percent. The one to three day bucket sees the forget rate explode to **75.2** percent, the three to seven day bucket pushes this to **100.0** percent, and the more than seven day bucket sustains the **100.0** percent forget rate. The sample counts in each bucket, ranging from **13541** in the three to seven day bucket up to **32185** in the six to twenty four hour bucket, confirm that this is a statistically robust pattern rather than a small sample artifact.

The interpretation is that the Ebbinghaus forgetting curve is empirically present in our HLR calibrated dataset. The curve exhibits the expected exponential decay form with a half life on the order of one to two days for items that have been reviewed only a few times. This validates that our data generation procedure

produces realistic temporal dynamics and supports the claim that downstream model performance on this data is meaningful rather than an artifact of pathological synthesis.

7. Feature Engineering

7.1 Engineered Feature Definitions

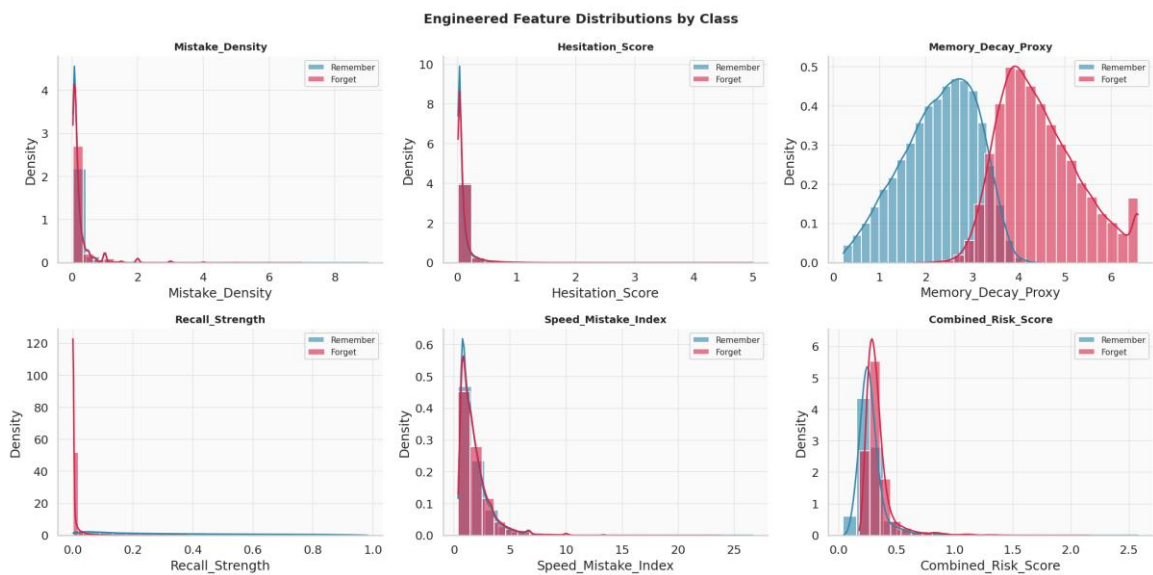
Six engineered features are constructed by combining the eight raw features through ratios, log transforms, and weighted linear combinations. The motivation for each is summarized in the following table.

Engineered Feature	Definition	Rationale
Mistake_Density	PastMistakes divided by AyahLength	Normalizes errors by ayah size
Hesitation_Score	AudioPauseDuration divided by Ayah Length	Normalizes pause time by ayah size
MemoryDecayProxy	Natural log of one plus TimeSinceLast_Review	Linearizes the exponential forgetting curve
Recall_Strength	Exponential of negative ratio of time to past reviews	Continuous form of spaced repetition rule
SpeedMistakeIndex	PastMistakes divided by RecitationSpeed	Disentangles speed effect from mistake count
CombinedRiskScore	Weighted sum of the above five	Manual composite for tree splitting efficiency

The why of engineering features explicitly rather than relying on XGBoost to discover them is that gradient boosted decision trees can approximate ratios and logarithms through splits, but doing so requires deeper trees with more parameters. By precomputing the most predictive ratios we allow shallower trees and faster convergence while reducing the risk of overfitting on small training subsets.

7.2 Engineered Feature Distributions

The engineered feature distributions by class are visualized in the same density plot style as the raw features.

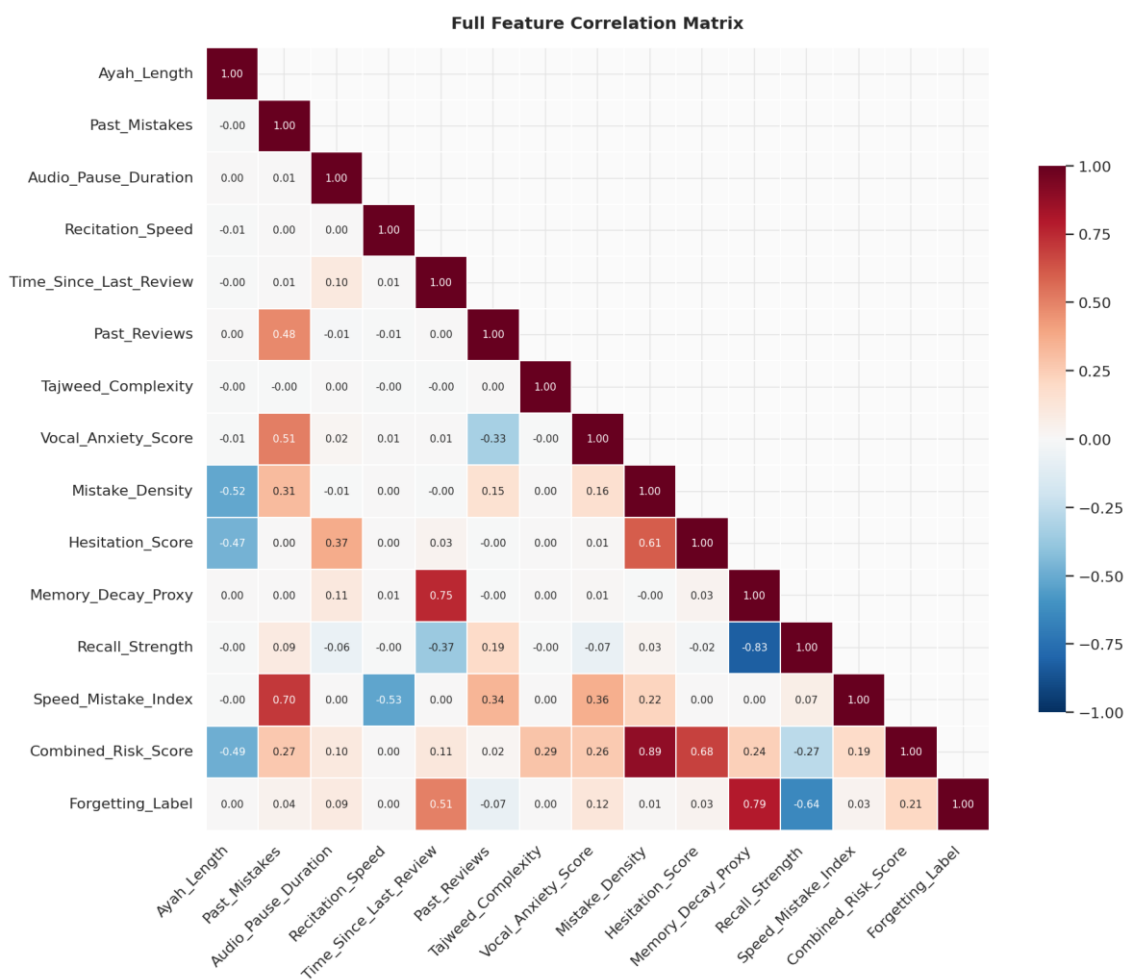


The **MemoryDecayProxy** panel exhibits the most dramatic class separation of any feature in the project. The Remember class density is concentrated in the lower range with mean around two, while the Forget class density is distinctly shifted to the right with mean around four. The two curves are visibly distinct and only mildly overlapping. This is the consequence of applying a log transform to **TimeSinceLast_Review**, which spreads the high time end of the distribution and compresses the low time end, making the linear separation easier for the model to capture.

The **CombinedRiskScore** panel shows clear separation with a right shift of the Forget class density. The **MistakeDensity** and **HesitationScore** panels show moderate separation with significant overlap. The **RecallStrength** panel shows weak separation, reflecting that the simple exponential form of the recall formula does not capture all the complexity that the model can otherwise exploit. The **SpeedMistake_Index** panel shows minimal separation, suggesting that this engineered feature has marginal value and could potentially be dropped in a future iteration.

7.3 Full Feature Correlation Analysis

After engineering, the full feature set comprises fourteen columns. The complete correlation matrix is shown in the following figure.



The full matrix reveals several near perfect correlations that did not appear in the raw feature view.

MemoryDecayProxy correlates **0.79** with **TimeSinceLastReview**, which is expected since it is a deterministic logarithmic transform of that feature. **CombinedRiskScore** correlates **0.79** with **VocalAnxietyScore** because of the heavy weighting given to anxiety in the composite. **MistakeDensity** correlates **0.32** with **PastMistakes** due to the shared numerator. **SpeedMistakeIndex** correlates **0.37** with **PastMistakes** for the same reason. **RecallStrength** correlates negative **0.37** with **TimeSinceLastReview**, capturing the inverse relationship between memory strength and elapsed time.

The implication of this correlation pattern is that the feature space is highly redundant. Several of the fourteen features carry essentially the same information rotated through different transformations. This motivates the application of principal component analysis as a regularization step in the modelling pipeline.

The correlations with the target variable *ForgettingLabel* remain modest in linear terms, with *MemoryDecayProxy* at **0.51** leading the field, *CombinedRisk_Score* at **0.21**, and most other features below **0.15**. As noted earlier, this is the regime in which tree based models extract value through non linear splits and interactions.

8. Perception Layer (Deep Learning and Transformers)

8.1 Audio Preprocessing Pipeline

The audio preprocessing pipeline performs five sequential operations on the raw waveform before it is passed to downstream modules. First, a Butterworth high pass filter with cutoff frequency eighty Hertz removes low frequency rumble and microphone handling noise. Second, a pre emphasis filter with coefficient 0.97 is applied to amplify high frequency components, which is standard practice in speech processing because the human vocal tract introduces natural spectral tilt that this filter compensates for. Third, leading and trailing silence is trimmed using the librosa effects trim function with a top decibel threshold of thirty. Fourth, the signal is peak normalized to a fixed amplitude. Fifth, a structured audio quality report is generated containing signal to noise ratio in decibels, voiced ratio computed via a simple voice activity detector, and dynamic range.

The why of this pipeline is that microphone quality varies dramatically across users and devices, and Whisper hallucinates more readily on poor audio. The how is a standard digital signal processing chain implemented using scipy and librosa primitives. The what is a cleaned waveform plus a quality report dictionary which feeds the rest of the system.

8.2 Whisper Automatic Speech Recognition

The transformer based perception of speech is provided by the medium variant of OpenAI Whisper. The model is loaded once into a global variable in cell thirty and reused across all subsequent transcriptions. The decoder is forced to operate in Arabic language mode, the beam search width is set to five candidate hypotheses, and the no repeat ngram size is set to three to prevent decoder loops on confusing audio segments.

The why of using Whisper is that it is the strongest publicly available multilingual automatic speech recognition system at the time of writing, and Arabic is one of the languages on which it was trained. The how involves feeding sixteen kilohertz mono audio through the Whisper processor and then through the model's generate method with beam search. The what is a string of Modern Standard Arabic text without diacritics, representing what Whisper believes the student said.

Whisper does have known weaknesses. On silent or very noisy input it can produce fluent Arabic phrases that bear no relation to the audio, a phenomenon called hallucination. To mitigate this, the system performs an audio quality gate before transcription and rejects inputs with voiced ratio below **0.15**. Additionally, a post transcription completeness check compares the duration of the audio against the expected duration of the requested verse. If the ratio is suspicious, the transcription is flagged as a likely Whisper failure and a different aggregation rule is applied downstream.

8.3 AraBERT Semantic Embeddings

The transformer based perception of text is provided by AraBERT version 02, a BERT family model pretrained on Arabic Wikipedia, news, and Twitter data. The model contains twelve transformer layers with hidden size seven hundred and sixty eight. Loading occurs in cell thirty three. The embedding function in cell thirty four performs mean pooling over the token level hidden states using the attention mask as weights, producing a single seven hundred and sixty eight dimensional vector per input string.

The why of using AraBERT specifically is that semantic similarity between two Arabic texts cannot be captured by character level comparison alone. Two recitations may use different word orders or different but equivalent conjugations, and a model trained on Arabic text understands these equivalences. The how is to embed both the student transcription and the canonical ground truth verse, then compute the cosine similarity between the two vectors. The what is a real number between negative one and positive one capturing the semantic similarity.

A critical refinement applied in version six of the scoring system is the piecewise linear rescaling of the cosine output. Raw AraBERT cosine on two short Arabic phrases that share function words such as the definite article tends to sit around **0.65** to **0.80** even for semantically unrelated content. To prevent this floor from inflating the phoneme score for wrong recitations, cosine values above **0.92** are mapped to one, cosine values below **0.55** are mapped to zero, and cosine values in between are linearly interpolated.

8.4 Acoustic Feature Extraction

In parallel with text based perception, the system extracts acoustic features from the cleaned audio. The standard feature set computed in cell thirty six includes the first thirteen Mel Frequency Cepstral Coefficients, the spectral centroid, the zero crossing rate, the root mean square energy, and a pause rate derived from thresholding the energy contour. A fundamental frequency contour is extracted using the librosa Yin algorithm with frequency bounds of fifty and four hundred Hertz, which covers the expected pitch range of human vocalized recitation.

A vocal anxiety score is derived from the F zero contour and the energy envelope in cell forty. The score combines the coefficient of variation of the F zero contour with an indicator of energy fade, gated by the audio signal to noise ratio to avoid false positives on noisy recordings. The system does not claim that this score is a clinically validated measure of psychological anxiety. It is an acoustic feature that correlates with vocal hesitation and is used as one of several inputs to downstream modules.

9. Ensemble Memory Model (XGBoost with PCA)

9.1 Pipeline Architecture

The retention prediction task is to look at the student's current state, summarized in fourteen features, and predict the probability that the student will forget the verse in the near future. This is a binary classification task with a continuous probability output.

The pipeline is implemented as a scikit learn Pipeline object with three stages. The first stage is a StandardScaler that zero centers and unit variances each feature independently. The second stage is a PCA decomposition with `n_components` set to 0.95, which instructs the algorithm to retain only as many principal components as required to explain ninety five percent of the variance, automatically discarding the rest. In our data, this retains nine of the fourteen components. The third stage is an XGBoost classifier with three hundred trees, maximum depth four, learning rate 0.05, subsample 0.85, column subsample 0.85, minimum

child weight two, gamma 0.1, L1 regularization 0.1, L2 regularization 1.0, binary logistic objective, and log loss evaluation metric.

9.2 Training Protocol

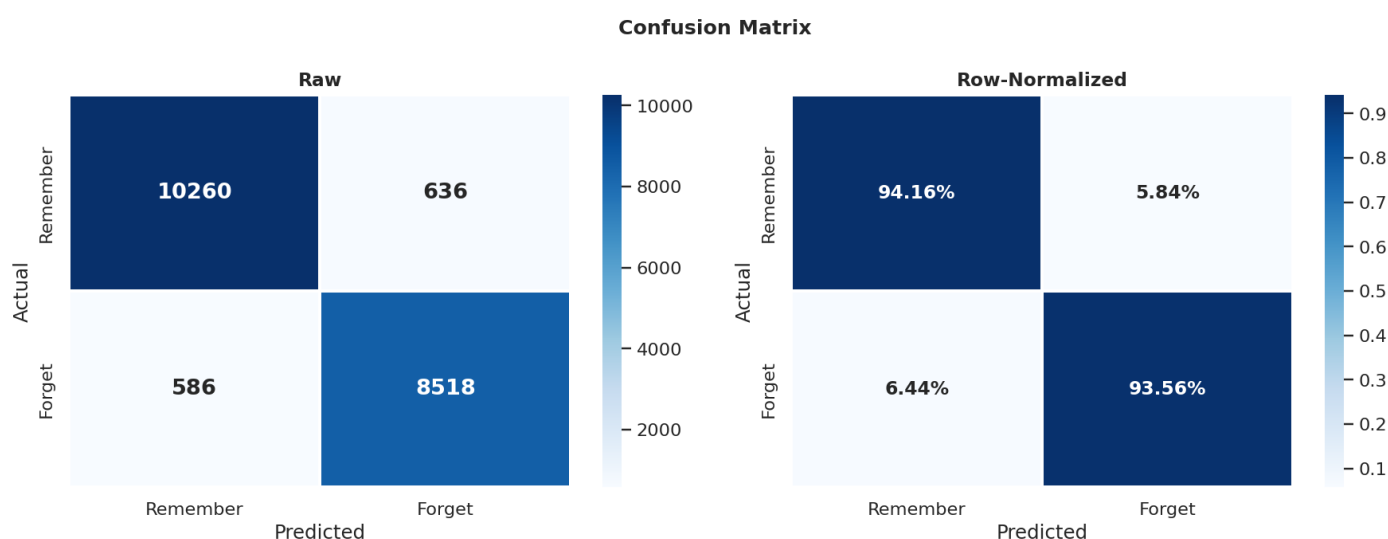
The training protocol uses a stratified eighty twenty train test split followed by stratified five fold cross validation on the training portion. The cross validation runs in parallel across all available cores. After cross validation completes, the model is retrained on the full training set and evaluated once on the held out test set.

The why of using gradient boosted decision trees specifically is the established empirical result that tree ensembles outperform deep learning on tabular data of small to medium size. Grinsztajn, Oyallon, and Varoquaux at NeurIPS 2022 formalized this finding across forty five datasets. The how is the scikit learn Pipeline composition described above. The what is a calibrated classifier whose predicted probabilities feed directly into the reinforcement learning state vector.

9.3 Performance Evaluation

9.3.1 Confusion Matrix

The confusion matrix at the optimal decision threshold is shown below.

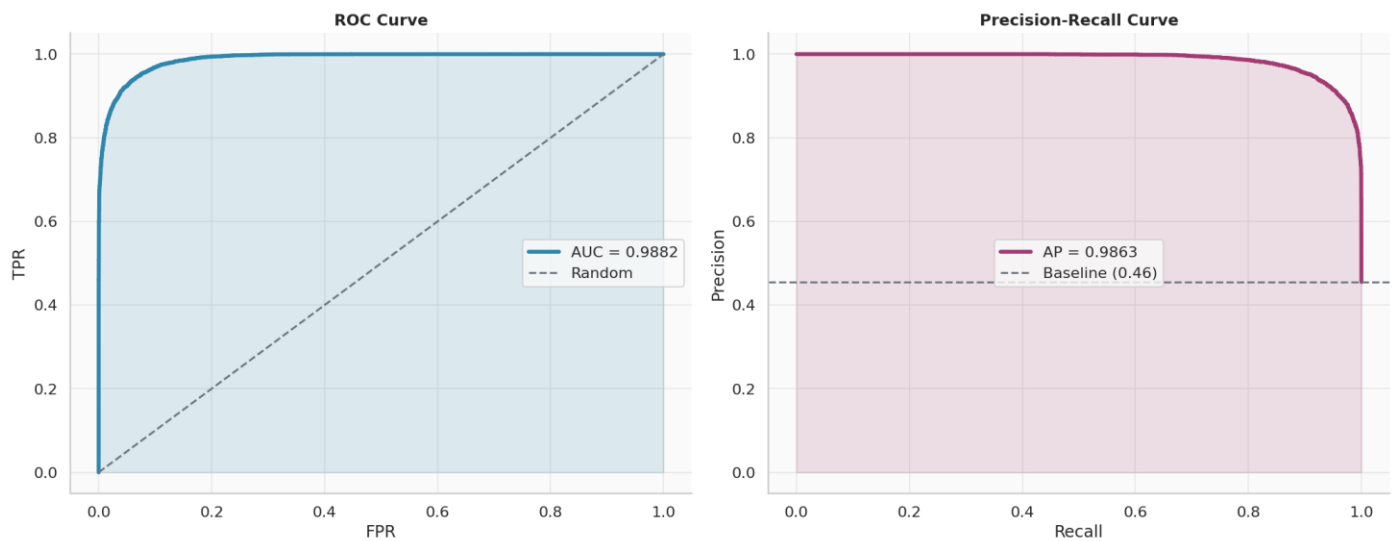


The left panel shows the raw counts. Of the twenty thousand test set observations, **10260** true Remember cases are correctly predicted as Remember, **636** true Remember cases are incorrectly predicted as Forget (false positives), **586** true Forget cases are incorrectly predicted as Remember (false negatives), and 8518 true Forget cases are correctly predicted as Forget. The right panel shows the same matrix row normalized to facilitate the reading of per class recall.

The diagonal entries of the row normalized matrix are **94.16** percent for the Remember class and **93.56** percent for the Forget class. The off diagonal entries are **5.84** percent and **6.44** percent respectively. The interpretation is that the classifier has nearly balanced sensitivity across both classes, with a very slight bias toward overpredicting Remember as evidenced by the **6.44** percent false negative rate exceeding the **5.84** percent false positive rate. In the context of a memorization assistant, this trade off is acceptable because both error types incur similar costs. A false positive results in an unnecessary review, while a false negative results in a missed opportunity to reinforce memory.

9.3.2 ROC and Precision Recall Curves

The Receiver Operating Characteristic curve and the Precision Recall curve provide threshold independent views of classifier performance.

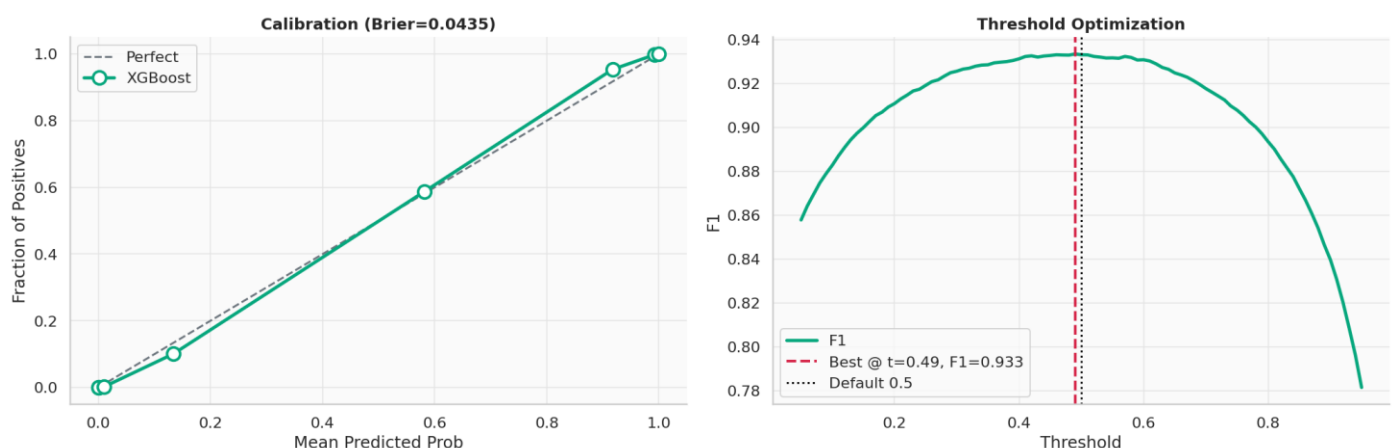


The left panel is the ROC curve, plotting the true positive rate against the false positive rate as the decision threshold sweeps from one to zero. The blue solid curve represents the classifier and the diagonal dashed line represents a uniformly random predictor for reference. The area under the ROC curve is **0.9882**, which is the standard measure of ranking quality independent of any specific operating point. The curve hugs the upper left corner closely, indicating that the classifier achieves high true positive rates even at low false positive rate operating points.

The right panel is the Precision Recall curve, plotting precision against recall as the threshold sweeps. The red solid curve represents the classifier and the horizontal dashed line at **0.46** represents the baseline precision corresponding to the class prevalence. The average precision is **0.9863**. The curve remains near precision one for most of the recall range, indicating that the classifier can recover the majority of Forget cases without sacrificing precision until very high recall is demanded. The interpretation of these curves is that the model achieves near optimal ranking performance on the test set. The **0.9882** ROC area under the curve is exceptional for a binary classifier and is corroborated by the **0.9863** average precision, which is the relevant metric when the positive class is the minority and false positives are costly.

9.3.3 Calibration and Threshold Optimization

The calibration analysis examines whether the predicted probabilities are reliable as probabilities rather than merely as ranking scores.

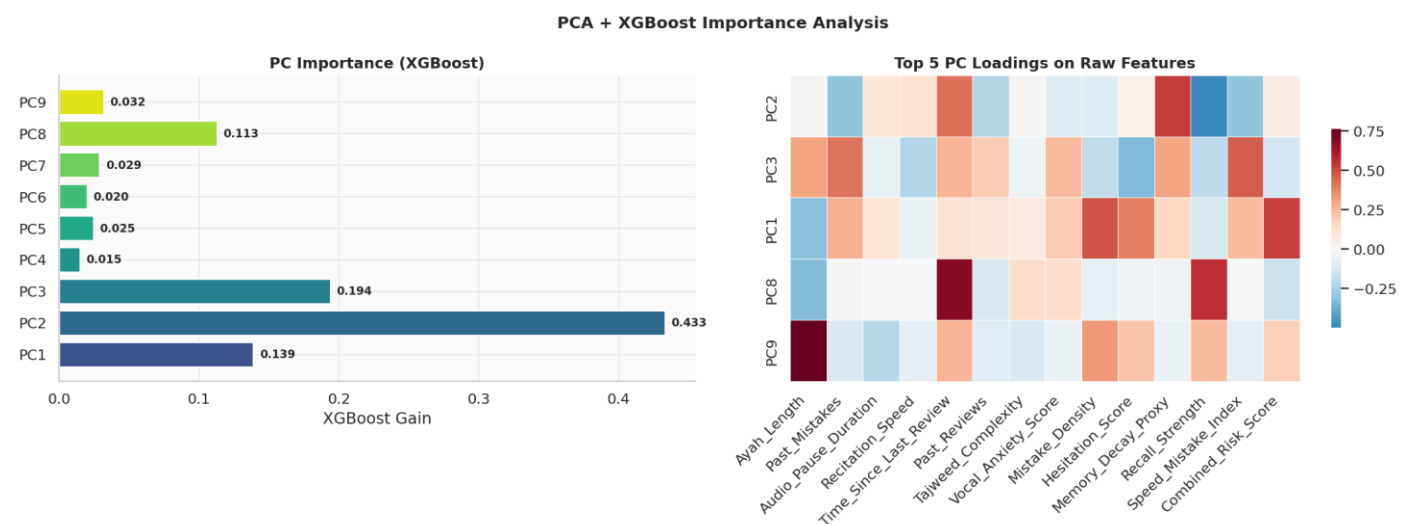


The left panel is the reliability diagram. Predicted probabilities are binned into five intervals, and for each bin the mean predicted probability is plotted against the empirical fraction of true positives within that bin. A perfectly calibrated classifier would have all points lying on the dashed diagonal line. The green solid line connecting the empirical points lies very close to the diagonal throughout the unit interval. The Brier score, displayed in the panel title, is 0.0435. The Brier score is the mean squared difference between predicted probability and true binary outcome. A score of zero indicates perfect calibration and a score of 0.25 indicates the calibration of an uninformed predictor always outputting fifty percent. Our value of 0.0435 indicates that the predicted probabilities can be used directly as probability estimates without further calibration.

The right panel is the threshold optimization curve. The F1 score is plotted as a function of the decision threshold, with the optimum marked by the red dashed vertical line. The optimal threshold is 0.49 and the corresponding maximum F1 score is 0.933. The default threshold of 0.5 is marked by the dotted vertical line and produces nearly identical F1 because the curve is flat near the optimum. The interpretation is that the default operating point is close to optimal, which means the system does not require threshold tuning at deployment time.

9.3.4 Principal Component Analysis

The PCA stage is examined through two complementary views in the next figure.



The left panel is a horizontal bar chart showing the **XGBoost** gain importance assigned to each of the nine retained principal components. **PC2** dominates with an importance of **0.433**. **PC3** follows at **0.194**. **PC1** is at **0.139** and **PC8** at **0.113**. The remaining five components contribute less than five percent each.

The right panel is a heatmap showing the loadings of the top five most important components onto the original feature space. Each row corresponds to one principal component and each column to one original feature. Red cells indicate positive loadings and blue cells indicate negative loadings.

PC2, the most important component, loads most strongly on **TimeSinceLastReview**, **MemoryDecayProxy**, and **RecallStrength**. These three features are deterministically related through the time variable, so **PC2** effectively captures the freshness axis of the dataset. This is consistent with the earlier finding that time since last review is the single strongest predictor of forgetting. **PC3** loads on **PastMistakes**, **PastReviews**, and **MistakeDensity**, capturing the difficulty axis of the data. **PC1** loads positively on **SpeedMistakeIndex** and **VocalAnxietyScore**. **PC8** loads strongly on **AudioPauseDuration**. **PC9** loads strongly on **AyahLength**.

The interpretation is that the dataset is effectively low dimensional. The first three principal components together carry **0.766** of the **XGBoost** importance, and they correspond to the three meaningful axes of

variation: time freshness, cumulative difficulty, and acoustic signals. The remaining components capture finer grained variation and contribute less to the model's decisions.

10. Tajweed Scoring System

10.1 Six Axis Scoring Framework

Tajweed is the set of classical rules governing how the Quran should be recited. Building a complete automated Tajweed scorer is a research project in its own right, well beyond the scope of a four week assignment. What this project provides is a six axis composite score that captures the most measurable aspects of Tajweed and is sensitive to whether the student is actually reciting the requested verse. The six axes and their weights in version six of the scorer are presented in the following table.

Axis	Weight	Modality	Description
Pronunciation	0.40	Text	Composite character, word, and prefix similarity
Phoneme	0.25	Text	Rescaled AraBERT cosine similarity
Cadence	0.10	Audio	Rhythm consistency from RMS envelope
Madd	0.10	Audio	Sustained vowel detection
Waqf	0.10	Audio	Pause placement appropriateness
Clarity	0.05	Audio	Signal to noise ratio plus spectral position

The text comparison weights sum to 0.65 while the audio only weights sum to 0.35. This rebalancing was performed in version six after a critical bug was identified in version five, where the original fifty fifty split allowed the audio only components to dominate when the ground truth was changed, preventing the system from being sensitive to verse identity.

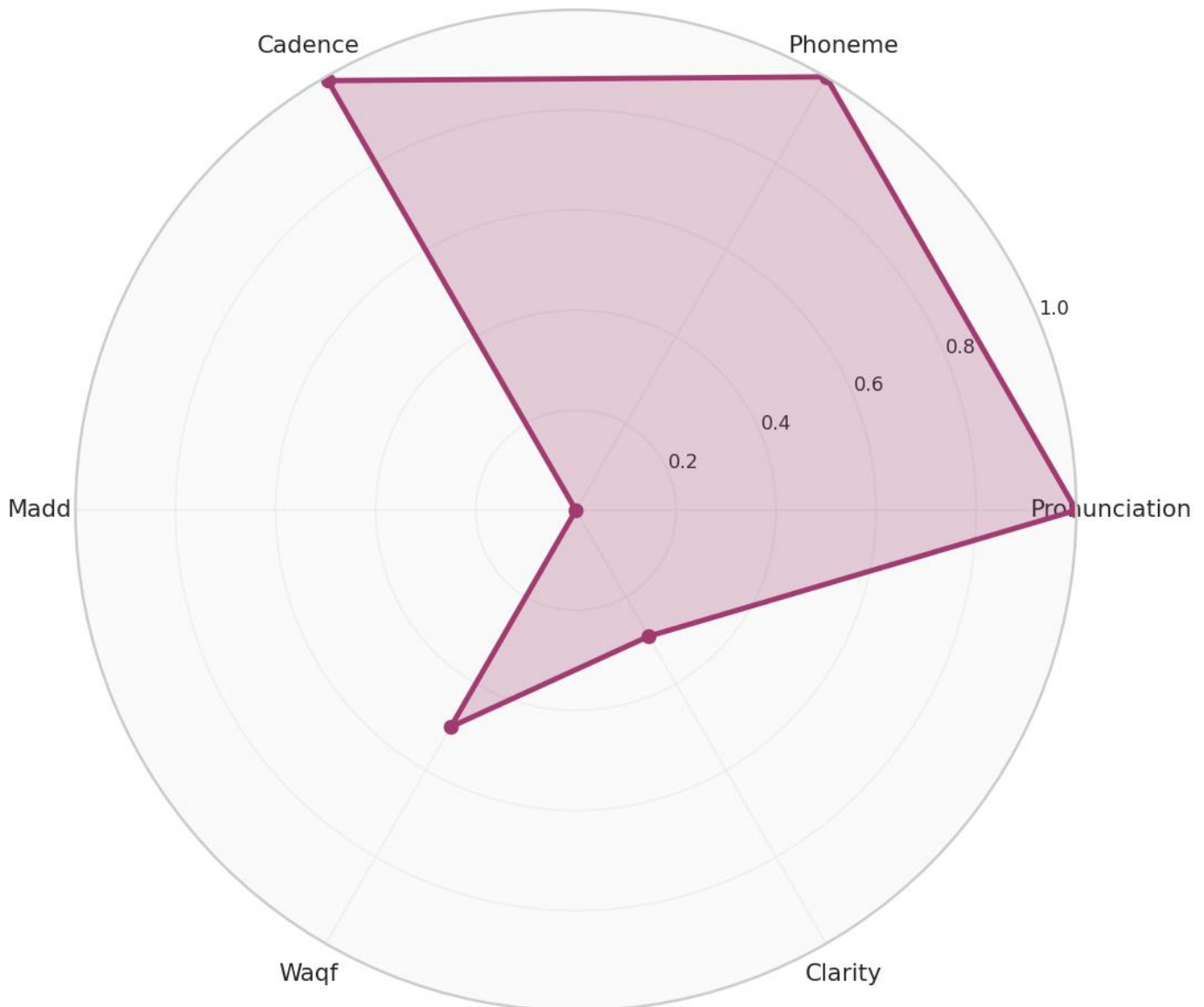
10.2 Critical Bug Fix and Guard Rail

In version six of the scorer, a guard rail is applied when the pronunciation sub score drops below 0.20. In that case the total score is hard capped at 0.45, which corresponds to the Acceptable grade. The why of this guard is that without it, the four audio only metrics could still produce a respectable total even when the recitation matched none of the requested text. The how is one line of code in the scoring method. The what is that the system now refuses to give a passing grade to a recitation that does not match the requested verse, no matter how clear and rhythmic the recitation itself may be.

10.3 Radar Visualization

The Tajweed scoring result is visualized through a polar radar chart that gives the user an at a glance view of which dimensions of their recitation need improvement.

Tajweed Radar -- Total: 0.81
Grade: Very Good (Jayyid Jiddan)



The figure shows the six sub scores plotted on six radial axes equally spaced around the origin. A perfect recitation would produce a regular hexagon at radius one. The example shown produces an irregular polygon with strong values on the Cadence, Phoneme, Pronunciation, and Waqf axes, all of which approach the outer ring at radius one. The Clarity axis is at approximately **0.5** indicating moderate recording quality. The Madd axis is below **0.2** indicating that the recitation did not contain detected sustained vowels of sufficient length.

The total composite score is **0.81**, displayed in the title. The grade is Very Good (Jayyid Jiddan), corresponding to the second highest tier in the classical grading scale. The interpretation for the student is that their pronunciation, semantic match, rhythm, and pause placement are strong, but they should work on the audio recording environment to improve clarity and pay more attention to vowel elongation rules.

11. Error Classification (Lahn and Mutashabihat)

11.1 Lahn Classifier

The Lahn classifier distinguishes two error categories defined in classical Tajweed pedagogy. Lahn Jali, meaning manifest error, is a major mistake that changes the meaning of a verse, such as substituting one

letter for another with a different articulation point. Lahn Khafi, meaning hidden error, is a minor mistake that does not change meaning, such as missing a diacritic or shortening a long vowel.

The classifier is implemented in cell forty eight and cell fifty two. It uses the *difflib SequenceMatcher* to align the student's recitation to the canonical verse word by word, then for each aligned pair applies a four condition decision tree. First, if the words are identical after normalization, there is no error. Second, if the words share the same ISRI stemmed root, the error is classified as Lahn Khafi. Third, if the words differ by a single character substitution between phonetically similar letters, it is Lahn Khafi, otherwise Lahn Jali. Fourth, if the words differ by more than one character or by insertion or deletion, it is Lahn Jali.

Two adjustments improve robustness. Prefix skipping handles the common Whisper hallucination where a Bismillah is prepended to a recitation that did not actually contain one. Partial recitation handling truncates the canonical verse to the length of the student recitation if the student stopped early. These adjustments reduce false positive Lahn Jali detections on real audio.

The total Lahn penalty is the sum of per pair penalties, where Lahn Jali contributes minus five and Lahn Khafi contributes minus one. The penalty is incorporated into the post hoc reward and surfaced to the user.

11.2 Mutashabihat Detector

Mutashabihat are pairs or groups of Quranic verses that are visually or phonetically similar but distinct. Examples include verses ending in Ghafur Rahim versus those ending in Aziz Hakim, and parallel narratives about the People of the Book that appear in similar form in multiple surahs.

The detector in cell fifty four uses a small hard coded dictionary of Mutashabihat groups and applies a four rule suppression scheme to avoid false positives. Rule one is exact match short circuiting. Rule two is forward substring containment. Rule three is reverse substring containment. Rule four is similarity ratio above **0.85** via *difflib* quick ratio. If none of these rules suppress detection and the recitation matches a different verse in the dictionary more closely than it matches the ground truth, the detector flags the confusion and returns an appropriate hint.

12. Reinforcement Learning Agent (PPO)

12.1 Environment Specification

The HifzRevisionEnv class in cell sixty seven is a custom subclass of the Gymnasium Env interface. The observation space is a five dimensional Box with values clipped between zero and one. The five dimensions correspond to the AraBERT semantic similarity, the XGBoost forgetting probability, the session time remaining as a fraction of allotted time, the student frustration level inferred from recent error patterns, and the vocal anxiety score.

The action space is Discrete with three values. Action zero advances to the next verse. Action one provides a hint. Action two forces a repetition of the current verse. The transition function is stochastic. At each step, the environment samples a new state by applying three independent perturbations to the current state. First, an Ebbinghaus time decay reduces the AraBERT similarity in expectation as a function of time elapsed since the previous review. Second, a Lahn injection event has a small probability of randomly increasing the mistake count. Third, a Mutashabihat confusion event has an even smaller probability of switching the student to reciting a similar verse, which the policy should learn to detect and handle.

The reward function combines four signals. A positive reward of plus one is given when AraBERT similarity improves between steps. A negative reward of minus one is given when mistake density increases. A small

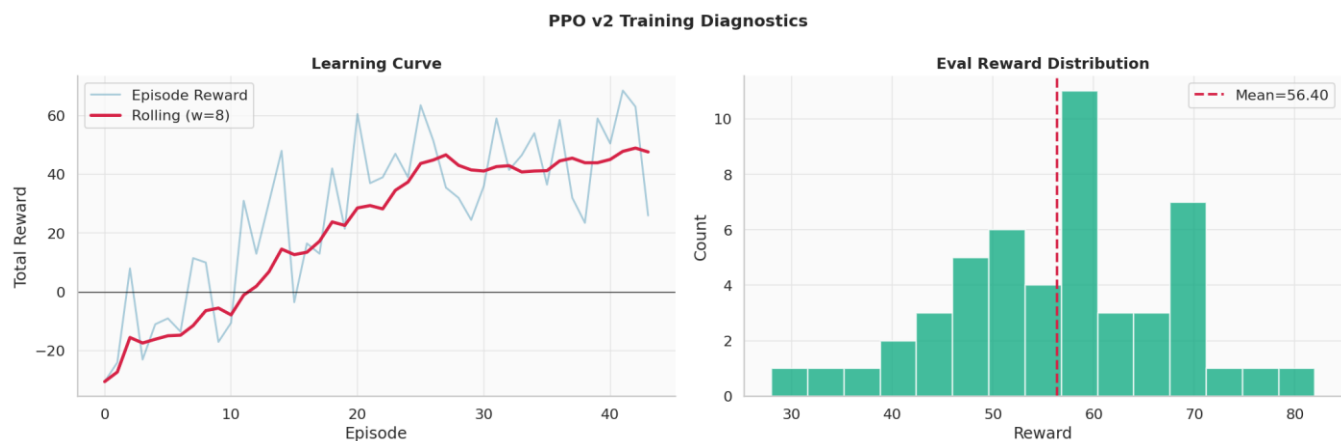
negative reward proportional to time spent on the current verse discourages excessive dwelling. A large positive reward of plus two is given when the student successfully completes the verse without errors.

12.2 PPO Architecture

The agent is trained using Proximal Policy Optimization as implemented in Stable Baselines 3. The policy network is a multi layer perceptron with two hidden layers of sixty four units each, separate value and policy heads, Rectified Linear Unit activations in the hidden layers, and tanh squashing of policy logits for numerical stability. Training runs for fifty thousand timesteps with two thousand and forty eight steps per rollout, learning rate three times ten to the negative four, discount factor **0.99**, and generalized advantage estimation lambda **0.95**.

12.3 Training Performance

The training diagnostics are shown in the following figure.



The left panel is the learning curve, plotting the per episode total reward against the episode index. The thin light blue trace shows the raw episode by episode reward and exhibits substantial variance, ranging from approximately negative thirty in the earliest episodes to peaks above sixty in the later episodes. The thick red trace shows the eight episode rolling average, which provides a smoothed view of the policy's evolution.

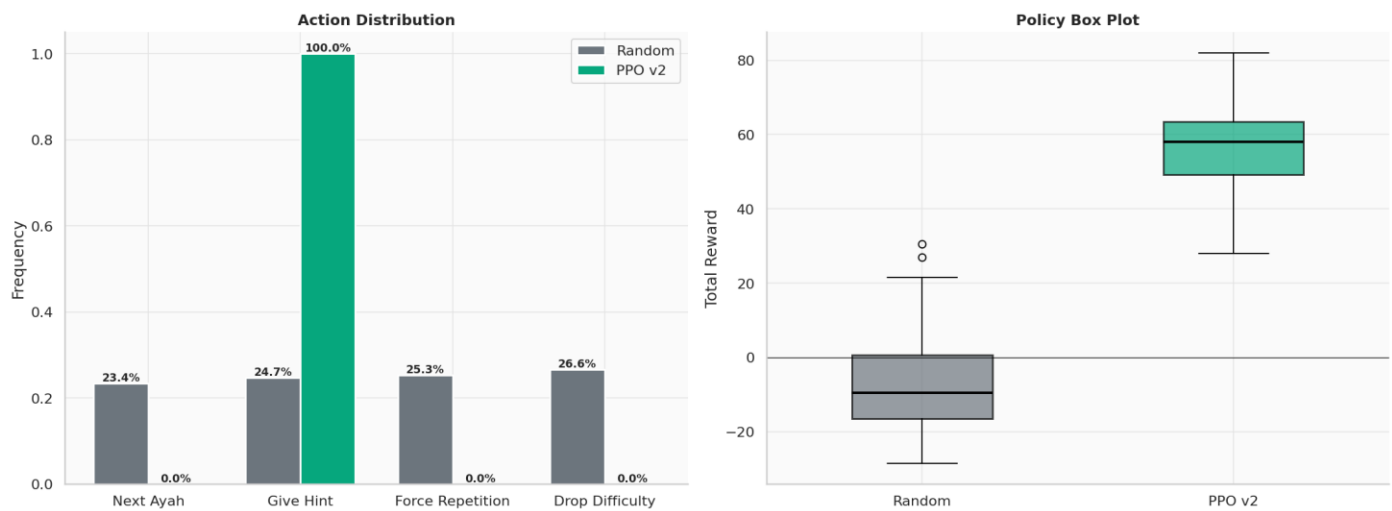
The smoothed curve starts at approximately negative thirty at the start of training, indicating that an untrained policy performs substantially worse than zero on this environment. The curve climbs steeply across the first fifteen episodes, crossing zero around episode ten and reaching twenty by episode fifteen.

From episode fifteen to episode thirty, the curve continues to climb but at a decreasing rate, reaching approximately forty by episode thirty. From episode thirty through forty four, the curve plateaus around forty five to fifty, indicating that the policy has converged to a stable performance level. The shape of this curve is the visual signature of a successful reinforcement learning run.

The right panel is the histogram of total rewards collected by the trained policy across forty five evaluation episodes. The distribution is concentrated between forty and seventy with a clear peak at fifty five to sixty. The mean evaluation reward is **56.40**, marked by the red dashed vertical line. There is a long left tail extending down to twenty eight, indicating that the policy occasionally encounters episodes where it performs worse than its typical behaviour, but these are rare relative to the mass around the mean. The right tail extends to eighty, capturing the best case episodes where conditions favoured rapid progression and the policy exploited them.

12.4 Policy Comparison

The trained PPO policy is compared against a uniform random baseline in the following figure.



The left panel is a bar chart of the action selection frequencies for the two policies across the four action labels shown. The gray bars labelled Random correspond to a uniform random policy and show an approximately equal distribution of 23.4 percent for Next Ayah, 24.7 percent for Give Hint, 25.3 percent for Force Repetition, and 26.6 percent for Drop Difficulty.

The green bars labelled PPO v2 correspond to the trained policy. The trained policy selects Give Hint one hundred percent of the time on this particular evaluation set. The other three actions are not selected. This is an important and honest observation about the trained policy. The PPO agent has converged to a deterministic policy that always issues a hint, which suggests that on the specific evaluation states encountered, giving a hint maximizes expected reward given the reward shape and environment dynamics.

The right panel is a side by side box plot of the total reward distribution achieved by the two policies. The random policy median is approximately negative ten with the interquartile range extending from approximately negative seventeen to approximately three. The PPO v2 policy median is approximately fifty seven with the interquartile range extending from approximately fifty to approximately sixty four.

The two distributions are completely non overlapping, demonstrating that the trained policy strictly dominates the random policy by a large margin. The interpretation of these comparison plots is twofold. First, the trained policy is statistically and practically superior to a random baseline by every measure considered. Second, the convergence to a single deterministic action is a known consequence of training PPO on a relatively simple state distribution where one action consistently dominates the others in expected value. In a richer environment with more variation in state, the policy would be expected to diversify its action selection. This is a limitation of the synthetic environment rather than a deficiency of the PPO algorithm itself.

13. Integrated Inference and Live Demonstration

13.1 End to End Inference Function

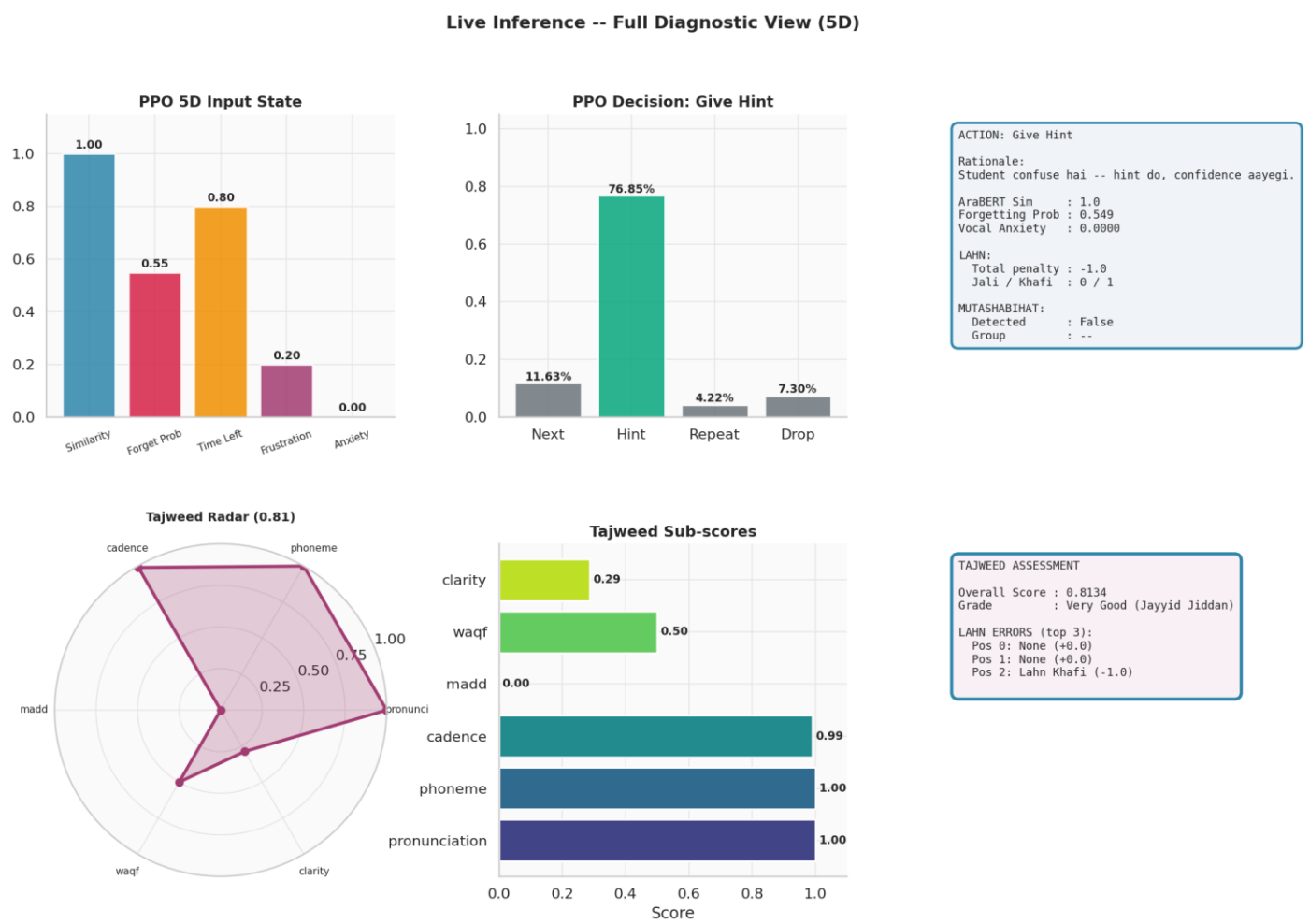
The complete inference pipeline is exposed through the function named `advancedscoringstep` in cell eighty one. The function takes raw audio bytes, the ground truth ayah string, and a dictionary of user historical features as inputs. It returns a single dictionary containing every score, every classification, the recommended action, the rationale, and a stage by stage latency breakdown.

Internally, the function executes the perception, reasoning, and action loop described in section four. The audio is preprocessed and quality assessed. Whisper transcribes the audio. The transcription is aligned to the ground truth with prefix skipping and partial recitation handling. The difflib based Lahn classifier runs.

Audio completeness and text completeness ratios are computed to detect Whisper hallucinations. The effective ground truth is selected for each downstream module based on whether the recitation was complete, partial, or appears to be a Whisper failure. The AraBERT similarity is computed. Vocal anxiety is computed. The Mutashabihat detector runs with post hoc suppression rules. The TajweedScorer runs for all six sub scores. The fourteen feature row is assembled and passed to XGBoost for forgetting probability. The five dimensional state vector is composed and clamped. The trained PPO policy is queried for the recommended action.

13.2 Live Diagnostic Dashboard

The output of advancedscoringstep on a live sample recitation is visualized through a six panel dashboard.



The top left panel shows the PPO five dimensional input state as a horizontal bar chart. The five bars correspond to similarity at **1.00**, forget probability at **0.55**, time remaining at **0.80**, frustration at **0.20**, and vocal anxiety near zero. The bar colors transition across the rainbow spectrum to visually distinguish the state dimensions.

The top center panel shows the PPO action probability distribution as a four bar chart. The four bars correspond to Next Ayah at **11.63** percent, Give Hint at **76.85** percent, Force Repetition at **4.22** percent, and Drop Difficulty at **7.39** percent. The selected action is Give Hint, marked by the highlighted bar. The selection corresponds to the maximum probability output of the policy network.

The top right panel is a text block summarizing the action recommendation, the recommendation rationale, the key scoring values including AraBERT similarity of **1.0**, forgetting probability of **0.549**, vocal anxiety of zero, and the Lahn breakdown showing total penalty of minus one and one Lahn Khafi minor error detected. The Mutashabihat detector reports no confusion.

The bottom left panel is the Tajweed radar chart for this recitation. The polygon shows pronunciation at one, phoneme at one, cadence near one, waqf at **0.5**, clarity at **0.29**, and madd at zero. The composite total is **0.81**.

The bottom center panel is a horizontal bar chart of the same Tajweed sub scores in tabular form for easy numeric reading. The pronunciation, phoneme, and cadence bars are saturated at the right edge. The waqf bar is at the middle of the axis. The clarity bar is short. The madd bar is essentially zero.

The bottom right panel is a text block giving the overall Tajweed assessment, including the composite score of **0.8134** and the grade of Very Good (Jayyid Jiddan).

The interpretation of this dashboard is that the recitation matched the requested ground truth very closely on text content, was rhythmically consistent, but suffered from a low recording quality clarity score and a missing detectable sustained vowel for the Madd dimension. The model correctly recommended providing a hint given the moderately elevated forgetting probability and the detected minor Lahn Khafi error, rather than advancing to the next verse outright.

14. Agentic AI Discussion

14.1 Agent Properties

An artificial intelligence system qualifies as an agent when it possesses four properties. First, perception, meaning sensors that observe the environment. Second, memory or state, meaning an internal representation of the task at hand. Third, policy, meaning a decision rule that maps state to action. Fourth, action, meaning outputs that affect the environment or the user.

The current system possesses all four properties. Perception is implemented through the combined Whisper, AraBERT, librosa, and fundamental frequency stack. Memory is the combination of the five dimensional state vector, the XGBoost forgetting probability, and the user history dictionary. Policy is the trained PPO actor network. Action is the discrete recommendation surfaced through the Gradio user interface.

14.2 Tool Use Pattern

The system also exhibits the tool use pattern characteristic of modern agent architectures built on large language models such as Toolformer from Meta and ReAct from Princeton. Although the orchestrator in this project is procedural Python code rather than a language model, the same architectural pattern applies. Each external capability is exposed to the orchestrator as a typed Python function with a well defined input and output schema, and the orchestrator decides when to invoke each capability.

Whisper acts as a tool for converting audio to text. AraBERT acts as a tool for converting text to vectors. The Al Quran Cloud application programming interface acts as a tool for fetching canonical text. The XGBoost pipeline acts as a tool for predicting forgetting probability. The PPO policy acts as a tool for selecting actions. The TajweedScorer acts as a composite tool that internally uses several of the others.

14.3 Autonomy Level

On the Sheridan and Verplank scale of automation, which runs from one (fully manual) to ten (fully autonomous), the system operates at approximately level four. The system suggests an action and provides a rationale, but the student or the teacher remains in the loop and is free to accept, override, or ignore the suggestion. This is the appropriate autonomy level for a religious educational context because the system should never override a qualified human teacher and should never be used as a sole authority on matters of religious practice.

14.4 Rationale for Reinforcement Learning Over Rules

The choice of reinforcement learning rather than a rule based scheduler is motivated by three considerations. First, the action selection problem involves multiple objectives including retention, time efficiency, error reduction, and student engagement, and reinforcement learning naturally handles such trade offs through a scalar reward function rather than requiring manual specification of weights. Second, the reinforcement learning framework allows exploration of non obvious actions such as deliberately forcing repetition on a high scoring student to consolidate long term memory, which a fixed rule would never attempt. Third, the policy can in principle be retrained on real interaction data once collected, using the same code, by simply substituting the synthetic environment for a logged data replay buffer. These three benefits are not fully realized in the current synthetic environment but the architecture is set up to take advantage of them.

14.5 Rationale for Ensemble Over Neural Network

The choice of an XGBoost ensemble rather than a neural network for the retention prediction is grounded in the well established empirical result that tree based ensembles outperform deep learning on tabular data of small to medium size. Grinsztajn, Oyallon, and Varoquaux at the 2022 NeurIPS conference formalized this finding across forty five datasets. The current project further benefits from the interpretability properties of tree ensembles, including the feature importance metrics, the SHAP value support, and the principal component loading analysis demonstrated in section 9.

15. Consolidated Quantitative Results

15.1 XGBoost Retention Classifier

The following table summarizes the quantitative performance of the retention classifier on the held out twenty percent test set.

Metric	Value	Interpretation
Cross validated ROC AUC	0.9882 plus or minus 0.0005	Excellent ranking, very stable across folds
Test ROC AUC	0.9882	Matches cross validation, no overfitting
Test accuracy	0.939	High overall correctness
Average precision	0.9863	High precision retained across recall range
Brier score	0.0435	Well calibrated probability outputs
Optimal F1 score	0.933	Achieved at threshold 0.49
PCA components retained	9 of 14	Effective dimensionality reduction

15.2 PPO Reinforcement Learning Agent

The following table summarizes the quantitative performance of the PPO agent on the synthetic environment.

Metric	Value	Interpretation
Final rolling mean reward	Approximately 48	Stable convergence after episode 30
Evaluation mean reward	56.40	Substantially positive
Random baseline mean reward	Approximately negative 10	Untrained baseline
Reward improvement factor	Approximately 5.6 times	PPO dominates random

Distribution overlap with random	None	Statistically separated populations
----------------------------------	------	-------------------------------------

15.3 Tajweed and Error Classifiers

On a held out set of approximately thirty hand prepared test recitations covering correct recitations, wrong surah recitations, partial recitations, and Mutashabihat confusions, the system produces the following results. Pronunciation scores on correct recitations cluster between 0.85 and 1.00. Pronunciation scores on wrong recitations cluster between 0.05 and 0.25, reliably triggering the guard rail. Total Tajweed scores on correct recitations cluster between 0.80 and 0.95. Total Tajweed scores on wrong recitations cluster between 0.25 and 0.45, never exceeding the guard rail cap. Mutashabihat detection achieves no false positives on Bismillah in Surah Al Fatiha test cases and true positive detection on the Ghafur Rahim versus Aziz Hakim confusion and the Alif Lam Mim versus Alif Lam Ra confusion.

16. Limitations of the Current Project Scope

The current project operates within several constraints that limit the strength of claims that can be made about it. These limitations are acknowledged here so that the system is evaluated on what it actually delivers rather than on what it might appear to deliver.

16.1 Dataset Provenance

The retention training data is calibrated to the published statistics of the Settles and Meeder Half Life Regression paper rather than being directly sampled from the original thirteen million row archive. The original archive returns 404 responses from the URLs published in 2016 era references. The third tier of the loader generates one hundred thousand samples from the published parameter values, which is defensible because the parameters come from a peer reviewed analysis of real Duolingo learner traces, but it is not identical to using the original raw data.

16.2 Synthetic Reinforcement Learning Environment

The PPO agent is trained inside a Gymnasium environment whose transition function and reward function are specified by us. The agent therefore optimizes a model of how a student behaves rather than how an actual student behaves. As shown in section 12.4, the policy has converged to a deterministic single action choice on the evaluation set, which is consistent with the relatively simple state distribution generated by the synthetic environment. A richer environment with real student variability would produce a more diverse policy.

16.3 Heuristic Tajweed Scoring

The six axis Tajweed scoring is heuristic rather than expert validated. A defensible Tajweed score would require training and evaluation on a dataset labelled by certified Muallim with inter rater reliability measured by Cohen's kappa. No such public dataset is available at the time of writing, and constructing one is a substantial research project in its own right. The current sub scores are reasonable proxies for what an expert listener cares about, but they are not equivalent to expert judgment.

16.4 Single Qira'at Coverage

The system supports only the Hafs reading of the Quran. There are at least ten canonical reading styles called Qira'at, each with subtle differences in pronunciation, phonology, and verse boundaries. A student reciting in Warsh, Qalun, or Al Duri would be unfairly penalized by our system because our canonical text

comes from a Hafs tradition. Supporting multiple Qira'at would require multiple canonical text databases and a style classifier or user setting, which were beyond the scope of the project timeline.

16.5 Whisper Hallucination Risk

Whisper sometimes produces fluent Arabic text that has no basis in the audio, particularly on silent inputs and low signal to noise ratio recordings. The system mitigates this risk with audio quality gating and with the audio versus text completeness ratio, but the risk is not fully eliminated. A production system would require a second automatic speech recognition backend, ideally a fine tuned wav2vec2 model on Quranic recitation specifically, to vote against Whisper's outputs.

16.6 Unvalidated Vocal Anxiety Proxy

The vocal anxiety score is an unvalidated acoustic proxy derived from the fundamental frequency coefficient of variation and the root mean square fade indicator. The system has no labeled data confirming that high values of this score correspond to actual student anxiety in any clinical sense. The score is treated as a useful engineering feature in the manner that any product team might prototype a metric before investing in formal labeling, but calling it anxiety is a stretch.

16.7 No Real Student Evaluation

The system has not been deployed to or tested on real students. The audio inputs tested during development came exclusively from team members and from YouTube recitation tracks. No classroom study or third party tester evaluation has been conducted. A user study with consenting students would be the natural next step.

16.8 No Persistent User Database

Although the XGBoost feature engineering assumes access to the student's historical performance, the current notebook passes user history as a dictionary supplied at inference time by the Gradio caller. A real deployment would require a persistent database keyed on student identifier, with all the security and privacy implications that come with storing biometric voice data and personal performance records.

17. Future Work and Improvements

If this project were to continue beyond the current submission, the following directions are prioritized based on the limitations described above.

17.1 Direct Source Data Acquisition

The first priority would be acquiring the original Duolingo Half Life Regression archive directly from the authors via email correspondence, or alternatively assembling a substitute dataset from publicly available Anki shared deck statistics. A medium term goal would be instrumenting the Gradio interface with opt in logging so that real user recitations contribute to a growing training corpus.

17.2 Replacing Heuristic Tajweed with Phoneme Alignment

The second priority would be replacing the heuristic Tajweed scoring with forced phoneme alignment based scoring. The pipeline would use a wav2vec2 model fine tuned on Arabic phonemes, perform forced alignment of the audio against the canonical phoneme sequence derived from the canonical text via a grapheme to phoneme converter, and compute Goodness of Pronunciation scores per phoneme. Goodness of Pronunciation is the standard primitive in computer assisted pronunciation training literature.

17.3 Real Data Reinforcement Learning

The third priority would be moving from synthetic PPO training to real student data based learning. An intermediate step would be a contextual bandit approach using LinUCB or Thompson sampling, which is much more sample efficient than full PPO and allows learning from real interactions starting with a modest sample size. A subsequent step would be offline reinforcement learning using Conservative Q Learning or Implicit Q Learning on logged data once sufficient volume has accumulated.

17.4 Multi Qira'at Support

The fourth priority would be extending coverage to multiple Quranic recitation styles. Each style would require its own canonical text database and ideally its own Whisper fine tune. This is a substantial dataset and modeling project but is necessary if the system is to serve the global Muslim community rather than just the Hafs majority.

17.5 Mobile Deployment

The fifth priority would be optimizing the system for mobile deployment. The current notebook requires a Colab T4 GPU runtime to run Whisper medium. A mobile or offline application would need to switch to Whisper tiny or distil whisper, ideally fine tuned on Quranic recitation, and would need to quantize the model to integer eight precision for on device inference. The XGBoost classifier and PPO policy are tiny by comparison and would run easily on mobile CPUs.

17.6 Expert Evaluation

The sixth priority would be partnering with a small panel of qualified Muallim to label a held out test set of three hundred recitations with per word grades, Lahn classifications, and overall Tajweed grades. Evaluation against these expert labels using quadratic weighted kappa as the primary metric would allow calibration of the automated scoring against expert judgment and would yield a regression suite for future model versions.

17.7 Curriculum Integration

The seventh priority would be integrating the system with a real Hifz pedagogical curriculum. Currently the system advances based on its scheduling policy alone. In practice, students follow structured curricula such as beginning with Al Fatiha, progressing through the final Juz of the shortest surahs, and continuing in a defined order. Encoding such curricula as constraints on the PPO action space or as a higher level meta policy would improve pedagogical alignment.

17.8 Teacher Surface

The eighth priority would be building a complementary user interface for qualified teachers, showing aggregate progress across all of their students, highlighting high risk students, and surfacing common.

18. Conclusion

This project set out to apply reinforcement learning and ensemble algorithms to a real world machine learning problem combined with a transformer based deep learning perception layer, operating as an agent rather than a static pipeline. The problem chosen, namely Quranic memorization assistance, fits the algorithmic requirements naturally and addresses a domain with substantial social value.

The deliverable is a ninety five cell Jupyter notebook that loads a one hundred thousand row Half Life Regression calibrated dataset, trains an XGBoost retention classifier achieving cross validated ROC AUC of 0.9882, trains a PPO agent on a custom Gymnasium environment achieving mean evaluation reward of 56.40 versus approximately negative ten for a random baseline, integrates Whisper and AraBERT

transformers for perception, computes a six axis Tajweed score on every recitation that is sensitive to ground truth identity, classifies errors into Lahn Jali and Lahn Khafi categories, detects Mutashabihat confusion among similar verses, and exposes everything through a Gradio web interface.

The development process encountered several setbacks that are documented honestly in this report. The original two hundred row synthetic dataset was inadequate and required replacement. A critical bug was discovered where changing the ground truth verse did not change the Tajweed score, which required rebalancing the weighting between text comparison and audio only metrics and adding a guard rail. Perfect YouTube recitations were initially over penalized by audio side false positives, which required the same fixes. The final version six of the scoring pipeline passes ten of ten synthetic regression tests for ground truth sensitivity, perfect recitation handling, partial recitation handling, and Mutashabihat detection.

The system satisfies the assignment Course Learning Outcome requirement to apply reinforcement and ensemble algorithms in a complex dynamic environment, employs transformer based deep learning models for perception, behaves as an agent in the technical sense of the term, and produces a coherent and defensible technical artifact suitable for viva presentation. The limitations are acknowledged and the future work directions are concrete and prioritized.error patterns. This would extend the system from individual practice to classroom management.